Conservation Status Governance — BA Spec

Path: `docs/planning/business-analysis/conservation-status-governance.md` **Date:** 2026-04-16 **Status:** Draft -- pending human review **Analyst:** Business Analyst Agent **Input Documents:** `CLAUDE.md` (Conservation status sourcing policy), `docs/planning/specs/gate-06-iucn-sync-seed-data.md`, `docs/planning/business-analysis/ba-assessment-v1.md` (Issue 4 review_status), `backend/species/models.py`, `backend/integration/tasks.py`, `backend/integration/models.py`, `todo.md` (Conservation status governance spec section) **Context:** Follow-up to gate 06. Gate 06 shipped the `Species.iucn_status` mirror + `ALLOW_IUCN_STATUS_OVERWRITE` setting + `iucn_sync` behavior that preserves existing `pending_review` rows. The four governance pieces below close the remaining gaps so that every mirror value is traceable to an authoritative or human-reviewed source.

---

1. Problem Statement

Gate 06 established the policy that `Species.iucn_status` is a denormalized mirror of the most-recent-accepted `ConservationAssessment`, never an independently editable field. The policy is enforced in one direction only: the IUCN sync correctly writes `iucn_status` from the assessment it just accepted. Four gaps remain:

1. **No audit trail.** There is no record of who changed what, when, or why. Today a Tier 5 admin could set `Species.iucn_status` directly in Django admin in violation of policy and no one would know; the `iucn_sync` itself leaves no per-species history beyond the `SyncJob` aggregate counters and the `last_sync_job` FK on the assessment row.
2. **No canonical source for human-reviewed status changes.** `ConservationAssessment.Source` has only `iucn_official` and `recommended_revision`. The policy forbids editing `Species.iucn_status` directly, which means operators have no legitimate path to record an expert override. This blocks legitimate workflows — notably, the CARES Coordinator flagging a Malagasy cichlid for reclassification based on field evidence — and makes the mirror policy impossible to honor in practice.
3. **No enforcement at the edit surface.** The policy lives in `CLAUDE.md` as a convention. The admin form does not stop an operator from editing the field directly, and the future GBIF / Darwin Core Archive export pipeline has no check that the mirror still agrees with the authoritative source it claims to reflect.
4. **No detection when IUCN and a manual assessment diverge.** `iucn_sync` preserves an existing `pending_review` row (gate 06 behavior), but it does not *create* a pending-review signal in the inverse case: a new IUCN category arriving that contradicts an existing `manual_expert` assessment. The sync will happily accept the IUCN row alongside the manual one with no conflict flag.

These gaps are load-bearing for the platform's credibility with conservation stakeholders. SHOAL, the IUCN SSC Freshwater Fish Specialist Group, and the ECA Workshop audience will all check whether status badges are traceable. "We mirror IUCN, except when we don't, and we can't tell you which is which" is not a viable story at the June 2026 workshop.

---

2. Stakeholders and Access-Tier Implications

Stakeholder	Role in governance	Tier

Tier 5 Administrator	Runs `iucn_sync`, approves `manual_expert` assessments, resolves conflicts, views audit log	5
Tier 3 Conservation Coordinator	Creates `recommended_revision` assessments, flags assessments for review, receives conflict alerts, sees audit entries for species they coordinate	3
Tier 2 Registered Researcher	Reads current accepted status only. Does not see `pending_review`, divergence flags, or audit history.	2
Tier 1 Public	Reads current accepted status only. Sees "Not yet assessed" when the mirror is null. Never sees conflict state.	1
`iucn_sync` system actor	Writes `iucn_official` assessments and mirror updates; writes audit entries as "system" actor	n/a

Tier implication for the audit log: Full read access is Tier 5 only. Tier 3 coordinators may need a scoped view of audit history for species within their institution's active breeding programs (to answer "when did this species' status change and why?"), but an unrestricted cross-species audit browser is admin-only. PM should decide whether to ship the scoped Tier 3 view in phase 1 or defer; see Requirement 1.

Tier implication for the conflict alert: Conflicts are resolved by Tier 5 (admin) with input from Tier 3 (coordinator). Tier 1-2 never see conflict state — they see only the current accepted category per the existing `ConservationAssessmentQuerySet.for_tier` semantics.

---

3. Requirements

The four requirements are dependency-ordered. Requirement 2 (add `manual_expert`) is the cheapest and unblocks the others. Requirement 1 (audit log) is the substrate all three other requirements write to. Requirements 3 and 4 both depend on both 1 and 2.

Requirement 1 — Audit log infrastructure

Need. Every change to `Species.iucn_status` and every create/update/delete on a `ConservationAssessment` row is recorded with: actor (user id or system source string), action (create/update/delete/mirror_write), before/after values of changed fields, timestamp, and operator-supplied reason (if applicable). Audit entries are append-only and immutable in the admin.

Recommendation: hand-rolled `AuditEntry` model, not an off-the-shelf package.

Justification:

- **Query pattern we actually need** is "show every change to this species' status with actor and reason, filterable by time range, across both `Species.iucn_status` and `ConservationAssessment` rows for that species." `django-simple-history` creates a shadow history table per model and encourages per-model queries; stitching a species-level view across `HistoricalSpecies` and `HistoricalConservationAssessment` is possible but awkward and couples the audit UI to the history-table schema. `django-auditlog` centralizes entries in one table (closer to what we want) but stores changes as a freeform JSON `changes` field that we'd still wrap for reliable querying, and its actor model assumes Django auth only — our system actor for `iucn_sync` is not a user.
- **Actor model fit.** We need to distinguish (a) authenticated users, (b) the `iucn_sync` system actor, (c) future sync actors (`fishbase_sync`, `gbif_publish`). A hand-rolled `AuditEntry` with an

`actor_type` enum (`user` / `system`) and optional `actor_user` FK + `actor_system` string is cleaner than fighting `auditlog` 's user-only assumption or `simple-history` 's request-coupled `history_user` .

- **Retention policy.** The audit table is the legal/governance record of conservation status decisions. Retention is indefinite by default. Off-the-shelf packages assume you might prune; we don't. A simple model with no auto-prune is the safest default.
- **Cost of retrofitting the 5 existing gates.** Gate 06 just shipped. `Species` and `ConservationAssessment` are the only models that need phase-1 coverage. Retrofitting an off-the-shelf package across `Species` , `ConservationAssessment` , plus the deferred `ExSituPopulation` / `Transfer` / `BreedingRecommendation` models later is a larger surface area than a hand-rolled model that only audits what we explicitly opt in.
- **What we give up.** `django-simple-history` gives you "read the model as it was at time T" for free. We don't need that. We need change feeds, not point-in-time reads. The cost of building the former on top of the latter when we need it is small; the cost of retrofitting the latter into the former is higher.

PM should confirm this recommendation before implementation; if there's appetite for time-travel reads later (e.g., "show this species' profile as it was on 2025-12-01"), `django-simple-history` becomes more attractive.

Phase 1 scope — what gets audited now:

- `Species.iucn_status` (field-level audit — other Species fields deferred)
- `ConservationAssessment` (row-level audit — all fields)

Phase 2 scope — explicitly deferred:

- `Institution` , `ExSituPopulation` , `HoldingRecord` , `BreedingEvent` , `Transfer` , `BreedingRecommendation` — these are valuable to audit but none are load-bearing for the conservation-status-governance story, and auditing them now expands the gate without helping the four pieces we're trying to deliver. Flag them as a post-MVP follow-up.
- Other `Species` fields (`cares_status` , `shoal_priority` , `endemic_status` , etc.). These have weaker governance requirements than `iucn_status` and can be added incrementally.

Acceptance Criteria:

Given a Tier 5 admin editing a `ConservationAssessment` and changing `review_status` from `accepted` to `pending_review` , *When* they save the form, *Then* an `AuditEntry` row is created with `target_type='ConservationAssessment'` , `target_id={the row id}` , `actor_type='user'` , `actor_user={admin user}` , `action='update'` , `before={'review_status': 'accepted'}` , `after={'review_status': 'pending_review'}` , `timestamp={now}` , and `reason={admin-supplied reason or empty string}` .

Given the `iucn_sync` task accepting a new IUCN category for a species and writing the mirror, *When* the mirror write occurs inside `_sync_one_species` , *Then* an `AuditEntry` row is created with `target_type='Species'` , `target_id={species id}` , `field='iucn_status'` , `actor_type='system'` , `actor_system='iucn_sync'` , `action='mirror_write'` , `before={old category or null}` , `after={new category}` , `reason='mirror from ConservationAssessment id={id}'` , and referenced `sync_job_id={job.id}` .

Given a Tier 5 admin viewing the `AuditEntry` list in Django admin, *When* the list renders, *Then* rows are ordered newest-first, filterable by `target_type` , `actor_type` , and date range, and the admin UI has no add/change/delete permissions (read-only exactly as `SyncJobAdmin` is today).

Given any request to view, edit, or delete an `AuditEntry` by a Tier 1-4 user (authenticated or anonymous), *When* the request is made through the API or admin, *Then* the response is 403 or the entries are not returned. Audit entries are Tier 5 only in phase 1.

Given a Tier 3 coordinator navigates to a species profile page in Django admin, *When* the page renders, *Then* an inline panel or linked view shows all `AuditEntry` rows where `target_type in ('Species', 'ConservationAssessment')` and target matches this species or its assessments, for the last 12 months, in reverse-chronological order. (PM decision: ship this in phase 1 or defer — see Open Questions.)

Given a developer runs `species.save(update_fields=['iucn_status', ...])` from somewhere other than the sanctioned mirror path, *When* the save occurs, *Then* the audit entry is still written (the audit hook is attached via `pre_save` / `post_save` signals or an explicit `AuditedSave` helper, not just to the sync task), so that policy violations surface in the audit log rather than being invisible.

Requirement 2 — Add `manual_expert` source to `ConservationAssessment.Source`

Need. Operators currently have no canonical way to record a human-reviewed status change. This is the prerequisite for Requirements 3 and 4 — "create a `ConservationAssessment` row instead of editing `iucn_status` directly" is meaningless advice if there's no valid `source` value for that row.

The `recommended_revision` source is semantically different: it captures "expert thinks this should be revised" without asserting the new category is authoritative yet (review_status typically stays `pending_review` or `under_revision`). `manual_expert` is "an expert has done the review and this is the current accepted category for this species in our system" — it can legitimately become the mirror source when no valid `iucn_official` assessment exists, or when a reconciliation decision (Requirement 4) overrides IUCN.

Enum change: `ConservationAssessment.Source` gains `MANUAL_EXPERT = "manual_expert"`.

Required fields when `source=manual_expert`:

Field	Currently	Required for manual_expert?	Rationale
`category`	required always	Yes (unchanged)	—
`assessor`	`blank=True`	**Yes** (conditionally required)	A human override without a named expert is not auditable.
`assessment_date`	`null=True`	**Yes** (conditionally required)	Needed to order manual vs iucn_official by recency.
`notes`	`blank=True`	**Yes** (conditionally required)	Supporting evidence / documentation narrative. Free text; citations encouraged.
`flagged_by`	`null=True`	**Yes** (conditionally required)	The User who created the row — serves as the "who entered this" for audit. (Note: `flagged_by` today is used in the existing Issue-4 review-flag flow; we reuse it here as "author of record" when `source=manual_expert`. PM should confirm the semantic stretch is acceptable vs

			adding a separate `created_by` FK. See Open Questions.)
`criteria`	`blank=True`	Optional	Not every manual override cites IUCN criteria.
`review_status`	default `accepted`	Defaults `under_revision` for manual_expert; must be explicitly moved to `accepted` by a second Tier 5 action	Avoids single-operator override without peer check. (See Open Questions — PM to decide on dual-approval requirement.)

Who can create a `manual_expert` row:

- Tier 3+ (Conservation Coordinator or higher) for species within their institutional scope
- Tier 5 (Administrator) unrestricted

Acceptance Criteria:

Given a Tier 3 coordinator creating a new `ConservationAssessment` via Django admin with `source=manual_expert`, *When* they submit the form with `category='CR'` but leave `assessor` blank, *Then* the form rejects the submission with a field-level error "`assessor` is required when `source` is `manual_expert`" and the row is not saved.

Given a Tier 3 coordinator successfully creating a `manual_expert` assessment with all required fields populated, *When* they save, *Then* the row is created with `review_status=under_revision` (not `accepted`) and `Species.iucn_status` is **not** yet mirrored from this row.

Given a Tier 5 admin reviewing an `under_revision` `manual_expert` assessment, *When* they change `review_status` to `accepted` and save, *Then* (a) the row is saved, (b) `Species.iucn_status` is updated to match `category` via the mirror path, (c) an `AuditEntry` is written for both the review_status change and the mirror write, (d) any existing `iucn_official` assessment for the same species transitions to `superseded` if its category disagrees (or the conflict is logged — see Requirement 4).

Given a Tier 2 researcher viewing a Species profile for a species with a `manual_expert` assessment in `accepted` state, *When* they view the status badge, *Then* the badge shows the `manual_expert` category and a small attribution indicator (e.g., "Expert review — [assessor name]") distinguishing it from an `iucn_official` source.

Given a species with a `manual_expert accepted` assessment dated 2026-03-01 and no `iucn_official` assessment, *When* `iucn_sync` runs and finds no IUCN record (species has no `iucn_taxon_id`), *Then* the `manual_expert` row is untouched, the mirror stays at its manual-expert value, and no error is raised.

Requirement 3 — Blocking warning on external-source contradiction

Need. Two enforcement points where the mirror policy must be enforced in UI/tooling, not just convention.

3a — Admin form validator on direct `Species.iucn_status` edit

Who sees the warning: Any user with edit rights to the Species admin (Tier 5 admins today; `SpeciesAdmin` is not tier-gated below 5 in phase 1).

What unblocks: Nothing. The edit is fully blocked. The admin form renders `iucn_status` as read-only with a help-text link: "To change the conservation status, create a new ConservationAssessment row for this species and set its review_status to accepted. See the Conservation status sourcing policy."

Rationale for hard-block rather than override-with-reason: Allowing an override with a reason field recreates the exact foot-gun the policy exists to prevent — `iucn_status` changes that aren't traceable through a ConservationAssessment. If the admin has a legitimate reason to change the status, the legitimate path (create a `manual_expert` row — Requirement 2) is now available. There is no case where direct edit is correct.

User-facing copy (admin form help text):

> *Conservation status is determined by the most-recent-accepted ConservationAssessment for this species. It cannot be edited here. To change it, create a new ConservationAssessment with `source=manual_expert` and have a Tier 5 admin set its `review_status` to `accepted`.*

Acceptance Criteria:

Given a Tier 5 admin viewing the Species change form in Django admin, *When* the form renders, *Then* the `iucn_status` field is displayed as read-only (disabled, not editable), shows the current value, and displays the help text above linking to the ConservationAssessment admin for this species.

Given a script or admin action attempting to POST a modified `iucn_status` value to the Species admin change view, *When* the request is processed, *Then* the server-side form ignores the submitted `iucn_status` value (field is in `readonly_fields`) and the value is not written, and a warning-level log entry is produced naming the user who attempted the bypass.

Given any code path calling `species.save()` or `species.save(update_fields=[...])` with a changed `iucn_status` from outside the sanctioned mirror helper, *When* the save is audited (Requirement 1), *Then* the `AuditEntry` `actor_system` is recorded as `unknown` rather than `iucn_sync`, making policy violations searchable in the audit log. (PM to confirm: do we also raise an assertion in DEBUG? See Open Questions.)

3b — Pre-publish check in GBIF / Darwin Core Archive export (gate 08+)

Who sees the warning: Tier 5 admin running the export (manually or via Celery). The GBIF publishing pipeline is deferred to gate 08+, so this is a requirement for that gate rather than for the immediate implementation; call it out now so gate 08 spec picks it up.

What unblocks: A single `--force` flag on the export command plus a mandatory `--override-reason` string. Using `--force` without `--override-reason` still fails. The override is recorded as an `AuditEntry` with `actor_user`, `action='override_publish_conflict'`, and the supplied reason.

Rationale for soft-block-with-override (vs hard-block like 3a): Publishing is a release event, not an edit. There are legitimate cases where the mirror diverges from IUCN intentionally (a reconciled manual override per Requirement 4), and we should not make every such species unpublishable forever. The override path ensures the divergence is acknowledged on every publish, recorded in audit, and visible to downstream reviewers.

User-facing copy (CLI error):

> *Refusing to publish: 3 species have `Species.iucn_status` that disagrees with their latest accepted `iucn_official` assessment:*
>
> - *Pachypanchax sakaramyi: mirror=CR, iucn_official=EN (assessment id 12345)*

> - *... If the divergence is intentional (e.g., a reconciled manual_expert override), re-run with `--force --override-reason="<explanation>"`.*

Acceptance Criteria:

Given the GBIF export command runs and finds species X with `iucn_status='CR'` but the latest accepted `iucn_official` assessment for X has `category='EN'`, *When* the pre-publish check runs, *Then* the export aborts with exit code non-zero, lists all such species, and no archive file is written.

Given the same scenario, *When* the command is re-run with `--force --override-reason="Manual override per 2026-03 expert review, see AuditEntry #442"`, *Then* the export proceeds, an `AuditEntry` is written with `action='override_publish_conflict'`, and the reason is included in the generated Darwin Core Archive's `eml.xml` `additionalInfo` section so downstream consumers see the divergence.

Given species X has an accepted `manual_expert` assessment and no `iucn_official` assessment, *When* the pre-publish check runs for species X, *Then* no divergence is flagged (there is no `iucn_official` to disagree with); the mirror is congruent with the `manual_expert` source and publishes without override. The `source` of the assessment is included in the DwC export metadata.

Requirement 4 — Inverse pending-review signal on IUCN vs. manual-expert conflict

Need. Today `iucn_sync` preserves existing `pending_review` rows (gate 06 behavior). It does not detect the inverse: an incoming IUCN category that contradicts an existing `manual_expert accepted` row for the same species. Per the mirror policy, silent coexistence of two disagreeing accepted assessments is forbidden.

When does a conflict exist:

The conflict trigger is **any `category` difference** between the incoming IUCN `category` and the existing `manual_expert` row in state `accepted` or `under_revision`.

Rationale against category-family-only trigger (e.g., "both threatened, no conflict"): the whole point of a `manual_expert` override is that the expert made a deliberate category-specific decision (e.g., CR vs EN is the difference between "immediate breeding program action" and "planned action"). Treating "both threatened" as congruent undermines the distinction the expert was making.

Note: A disagreement between an incoming `iucn_official` and an existing `iucn_official` (same source, new year) is NOT a conflict — that's just an updated assessment, which gate 06 already handles via upsert on `(species, source='iucn_official')`.

State transition on conflict detection:

1. The **incoming IUCN row is still written**, but with `review_status='pending_review'` instead of `accepted`.
2. The existing `manual_expert` row is untouched.
3. `Species.iucn_status` is **not** overwritten. Mirror stays at the current (manual_expert) value.
4. A `ConservationStatusConflict` record is created (new lightweight model, or a row in an existing alerts table — PM decision) capturing: species FK, manual_assessment FK, iucn_assessment FK, detected_at, status (`open` / `resolved`), resolution (nullable enum: `accepted_iucn` / `retained_manual` / `reconciled` / `dismissed`), resolution_reason, resolved_by, resolved_at.

5. An `AuditEntry` is written with `action='conflict_detected'` referencing both assessment ids.

Alert surface:

- **In-admin:** Tier 5 admin dashboard shows an "Open status conflicts" count; the `ConservationStatusConflict` admin view lists open conflicts sorted by `detected_at` desc.
- **Email/notification:** out of scope for phase 1 — admins check the admin dashboard. (If the platform already has a notification layer at gate 08, revisit.)

Who resolves it: Tier 5 admin, with input from the originating Tier 3 coordinator for the `manual_expert` row (when attributable via `flagged_by` /author FK).

Resolution outcomes:

Outcome	What happens to the manual_expert row	What happens to the iucn_official row	What happens to `Species.iucn_status`
`accepted_iucn`	transitions to `superseded`; `review_notes` records "superseded by IUCN 2026-xx-xx review"	transitions to `accepted`	overwritten to IUCN category via mirror path
`retained_manual`	unchanged (stays `accepted`)	stays `pending_review` indefinitely; `review_notes` records "divergent from manual_expert, retained per admin decision on YYYY-MM-DD"; **not silently accepted**	unchanged (stays at manual category)
`reconciled`	transitions to `superseded`	transitions to `superseded`; admin creates a **new** `manual_expert` row with a reconciled category and `accepted` status	overwritten to the new reconciled category
`dismissed`	unchanged	deleted (the conflict was a data error — e.g., IUCN taxon ID mapped to wrong species)	unchanged

Preventing re-fire on next sync:

The open question is: once an admin selects `retained_manual`, what stops the *next* week's IUCN sync from re-detecting the same conflict and creating another `ConservationStatusConflict` row?

Proposed mechanism: add a `conflict_acknowledged_assessment_ids` JSON field to the `manual_expert` assessment row (or equivalent on the Species), recording IUCN assessment ids (not categories) that have already been reviewed and retained. On each sync, the conflict detector compares (manual_expert.category vs iucn_official.category) AND (incoming iucn assessment_id NOT IN acknowledged list). If either is false, no new conflict is raised.

This keys on assessment identity, not category, because a new IUCN re-assessment — even one that still disagrees — warrants a fresh human review; a dormant unchanged assessment does not.

Acceptance Criteria:

Given species X has an `accepted manual_expert` row with `category='CR'` and `iucn_sync` pulls an IUCN assessment with `category='EN'`, *When* the sync processes species X, *Then* (a) a new `ConservationAssessment` row is created with `source=iucn_official`, `category='EN'`, `review_status='pending_review'`, (b) a `ConservationStatusConflict` row is created with `status=open` and references both assessments, (c) `Species.iucn_status` remains `CR`, (d) no mirror write or audit-mirror-write entry is produced.

Given the admin resolves the above conflict with `resolution=retained_manual` and `resolution_reason="Ziegler 2024 field data supports CR, IUCN uses outdated 2018 data"`, *When* the resolution is saved, *Then* (a) the conflict status flips to `resolved`, (b) the iucn_official row stays at `pending_review` with a note appended, (c) the manual_expert row's `conflict_acknowledged_assessment_ids` gains the IUCN assessment id, (d) an `AuditEntry` is written for the resolution.

Given the next weekly `iucn_sync` runs and the IUCN API returns the same assessment (unchanged assessment_id), *When* the sync processes species X, *Then* no new `ConservationStatusConflict` row is created, no category change occurs, and `records_skipped` increments.

Given the sync run after that retrieves a new IUCN assessment for species X with a new assessment_id (even if still `category='EN'`), *When* the sync processes species X, *Then* a new `ConservationStatusConflict` is raised because the acknowledgement was keyed on the prior assessment_id, not on category.

Given a species with an `accepted iucn_official` row at `category='EN'` and no manual_expert row, *When* the sync retrieves an updated IUCN assessment with `category='CR'`, *Then* this is the normal gate-06 path: the existing row is updated, mirror writes, no conflict is raised. (Confirms Requirement 4 does not regress gate-06 behavior.)

Given a Tier 5 admin viewing the Species profile in admin for a species with an open conflict, *When* the page renders, *Then* a visible banner says "Open conservation status conflict — IUCN says EN, manual expert says CR" with a link to the resolution form.

---

4. Interactions Between the Four Pieces

The requirements are not independent. Sequence and dependency:

1. **Requirement 2 (add `manual_expert`) must land first.** Without it, there is no legitimate target for Requirement 3a's "create a ConservationAssessment instead" redirect, and Requirement 4's conflict model has no "manual" side to conflict with.
2. **Requirement 1 (audit log) must land before or alongside Requirements 3 and 4.** Every blocking warning, every conflict transition, every override is supposed to produce an audit entry. Shipping 3 or 4 without audit infrastructure means the governance signals are fire-and-forget.
3. **Requirement 3a (admin read-only) can ship independently after 2.** It's a small admin-form change and does not depend on 4.
4. **Requirement 3b (GBIF pre-publish check) is gated on gate 08+ (GBIF publishing itself).** Call it out now so gate 08 spec picks it up; do not block the current governance gate on it.
5. **Requirement 4 (conflict detection) depends on 1 and 2.** It is the most complex piece and has the most PM-decision surface (see Open Questions).

Recommended implementation order: 2 → 1 → 3a → 4 → 3b (gate 08+).

---

5. Out of Scope / Deferred

- **Auditing models other than `Species.iucn_status` and `ConservationAssessment`.** `Institution`, `ExSituPopulation`, `HoldingRecord`, `BreedingEvent`, `Transfer`, `BreedingRecommendation`, and `SpeciesLocality` changes are not audited in phase 1. These matter but do not belong to the conservation-status-governance story.
- **Auditing other `Species` fields.** `cares_status`, `shoal_priority`, `endemic_status`, `taxonomic_status`, `in_captivity`, etc. are not audited in phase 1. The governance concern is specific to `iucn_status` because that's the mirror.
- **Point-in-time reads.** "Show me this species' profile as of 2025-12-01" is not a requirement. If it becomes one, revisit the `django-simple-history` recommendation.
- **Email/SMS/Slack notifications on conflicts.** Phase 1 relies on the admin dashboard. Notification plumbing is post-MVP.
- **Tier 3 scoped audit browser (cross-species).** PM may choose to defer this; Tier 5 full visibility is the baseline requirement.
- **Automated resolution suggestions.** The conflict resolution UI does not recommend which outcome to choose. Humans decide.
- **Retention / pruning of audit entries.** Audit is append-only and indefinite in phase 1.
- **GBIF export pre-publish check (Requirement 3b).** Deferred to gate 08+ but spec'd here so the gate-08 work picks it up.

---

6. Open Questions for PM

1. **Audit log implementation — hand-rolled vs `django-simple-history`.** BA recommends hand-rolled. If PM wants point-in-time reads later, `django-simple-history` becomes competitive. Confirm the recommendation or request a deeper comparison with explicit tradeoffs.
2. **Tier 3 scoped audit view — phase 1 or defer.** Scoped inline panel on species admin showing last-12-months audit entries for that species. Low implementation cost on top of the Tier 5 baseline, high value for coordinator workflows. PM call.
3. **Dual-approval for `manual_expert` → `accepted` transition.** BA's draft specifies that `manual_expert` rows default to `under_revision` and require a separate Tier 5 action to mark `accepted`. This prevents a single operator from unilaterally overriding IUCN. Is that friction acceptable, or should the creating user be able to create-and-accept in one step if they're Tier 5?
4. **Reuse of `flagged_by` as `manual_expert` author, vs adding a `created_by` FK.** The existing `flagged_by` field (added per BA Issue 4 for review-flag workflow) is semantically "who raised the review flag." Using it to mean "who authored the manual_expert row" is a stretch. Adding a separate `created_by` FK is cleaner. Migration cost is trivial. Confirm preference.
5. **Should `AuditEntry` include a `reason` prompt on every Species/ConservationAssessment admin save, or only on conflict-related actions?** Always-prompt is heavyweight but most auditable. Prompt-only-on-conflict is lighter but loses the "why" for routine edits. BA leans toward always-prompt for `manual_expert` saves and conflict resolutions only; not for IUCN-sync writes.
6. **`DEBUG`-mode assertion on direct `iucn_status` writes.** Requirement 3a records bypass attempts in audit; BA would additionally raise an assertion in `DEBUG` so developers see the violation immediately. Acceptable or too aggressive?
7. **`ConservationStatusConflict` as a standalone model vs a new `review_status` state / field on `ConservationAssessment`.** Standalone model is cleaner for query patterns ("show me all open conflicts") and for the resolution lifecycle. Field-on-assessment is lighter but awkward because the conflict is between two assessments, not a property of one. BA recommends standalone model; confirm.

8. **Conflict acknowledgement keyed on IUCN `assessment_id`.** Proposed mechanism assumes IUCN returns stable assessment IDs across syncs. Spot-check the IUCN API v4 response to confirm; if IDs change, we need a different idempotency key (e.g., `(iucn_taxon_id, year_published, category)` tuple).
9. **What is the user-facing rendering when a species has an open conflict but a Tier 1 public user visits the profile?** Per tier semantics, Tier 1 sees the current accepted category (the manual_expert one, since the IUCN row was forced to `pending_review`). But the status badge claims to mirror IUCN. Do we change the badge attribution to "Expert review" whenever the mirror's source assessment is `manual_expert`, or always say "IUCN" and footnote the source? Decision has UX implications beyond this gate.

---

7. Dependencies on Prior Gates

- **Gate 03 (Auth):** `AuditEntry.actor_user` FK depends on `User` model. Already in place.
- **Gate 04 (Django Admin):** admin form infrastructure for `SpeciesAdmin` read-only treatment. Admin already stood up.
- **Gate 06 (IUCN Sync):** `ALLOW_IUCN_STATUS_OVERWRITE` setting, mirror path in `iucn_sync._sync_one_species`, `SyncJob` model, `ConservationAssessment.source` and `review_status` fields. All in place on `gate/06-iucn-sync-seed-data` branch. Requirement 4's conflict detector is the natural extension point inside `_sync_one_species` — the existing branch that skips on `review_status=pending_review` becomes one of three branches (skip, update, conflict-detected).
- **Gate 08+ (GBIF Publishing — not yet spec'd):** Requirement 3b's pre-publish check lives here. Needs to be picked up in the gate-08 spec.

No prior gates need to be reopened. All four requirements can land on a new feature branch off `main` after the current `gate/06` branch merges.

---

8. Decisions (2026-04-16)

Answers to Section 6 open questions from the project owner:

1. **Audit log:** Hand-rolled `AuditEntry` model. No point-in-time reads needed.
2. **Tier 3 scoped audit view:** Ship in phase 1 (inline panel on species admin, last 12 months).
3. **Dual-approval for `manual_expert` → `accepted`:** Not required. Tier 5 can create-and-accept in one step. (Overrides the draft's `under_revision` default for manual_expert.)
4. **Authorship field:** Add separate `created_by` FK to `ConservationAssessment`. Do not overload `flagged_by`.
5. **Conflict model:** Standalone `ConservationStatusConflict` model.
6. **`reason` prompt scope:** Prompt only on `manual_expert` saves and conflict resolutions. Not for routine edits or IUCN sync writes.
7. **`DEBUG` assertion on direct `iucn_status` writes:** Acceptable. Raise `AssertionError` in DEBUG when a save touches `iucn_status` outside the sanctioned mirror path.
8. **IUCN `assessment_id` stability:** Verify against live API v4 now before PM breakdown. If IDs are unstable, substitute a composite idempotency key.
9. **Public badge attribution:** Show "Expert review" (Option a) when the mirror's source assessment is `manual_expert`. Standard IUCN attribution otherwise.

These decisions supersede any conflicting draft language above (notably the draft's dual-approval default in Requirement 2's table, which is withdrawn per decision 3).

End of BA spec. Ready for PM review and gate breakdown.